

Лабораторная работа №2. Работа с GitFlic и расширениями Git в Visual Studio Code

Цель работы

Ознакомиться с основными командами Git, научиться создавать локальный репозиторий, подключать его к удалённому репозиторию (на примере GitFlic) и работать с Git через командную строку и Visual Studio Code.

Краткие сведения о Git

Git — это распределённая система контроля версий, созданная Линусом Торвальдсом для разработки ядра **Linux**. В настоящее время **Git** используется для проектов любого типа и масштаба.

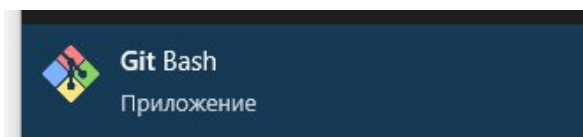
Система контроля версий позволяет:

- отслеживать изменения файлов;
- возвращаться к предыдущим версиям проекта;
- работать над проектом совместно с другими разработчиками;
- хранить проект как локально, так и в удалённом репозитории.

В данной лабораторной работе в качестве сервиса удалённого репозитория рассматривается **GitFlic** — российский аналог **GitHub**. При этом вы можете использовать любой привычный вам сервис (**GitHub**, **GitLab** и т.д.), так как принципы работы у них схожи.

Шаг 1. Проверка установленного Git

1. Запустите **Git Bash** на компьютере.



2. Выполните команду для проверки версии Git:

git version

Если Git установлен корректно, в терминале отобразится его версия:

```
palto$ git version
git version 2.52.0.windows.1
```

Шаг 2. Настройка пользователя Git

Перед началом работы необходимо указать имя пользователя и адрес электронной почты. Эти данные используются для подписи коммитов.

Выполните команды:

git config --global user.name "ваш_ник"

git config --global user.email "ваша_почта@yandex.ru"

```
Paltos@Paltosik92 MINGW64 ~  
$ git config --global user.name Paltosik92  
  
Paltos@Paltosik92 MINGW64 ~  
$ git config --global user.email denis.leontev92@gmail.com
```

Параметр **--global** означает, что настройки будут применяться ко всем Git-репозиториям на данном компьютере.

Шаг 3. Создание структуры проекта

1. Перейдите в папку **Документы**:

cd Documents/

```
Paltos@Paltosik92 MINGW64 ~  
$ cd Documents/  
  
Paltos@Paltosik92 MINGW64 ~/Documents  
$
```

2. Создайте папку с названием, содержащим номер лабораторной и ваше ФИО:

mkdir Lab2_ISRPO_FIO

3. Убедитесь в создании папки с помощью команды **ls**:

```
Paltos@Paltosik92 MINGW64 ~/Documents  
$ ls  
Blender/                               Lab5_Html_Leontev/  
'CD Projekt Red'/                     'My Games'/  
Fullstack/                             PowerShell/  
Html_learn/                            'Rockstar Games'/  
IISExpress/                            'SQL Server Manageme  
Lab1_ISRPO_Leontev/                   'SQL Server Manageme  
Lab2_ISRPO_Leontev/                   'Visual Studio 18'/
```

4. Перейдите в неё командой **cd** и в ней создайте папку **backend**:

cd Lab2_ISRPO_FIO

mkdir backend

```
Paltos@Paltosik92 MINGW64 ~/Documents  
$ cd Lab2_ISRPO_Leontev/  
  
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev  
$ mkdir backend
```

5. Убедитесь, что папка **backend** создана:

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev
$ ls
backend/
```

6. Перейдите в папку и проверьте текущий путь командой **pwd**:

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev
$ cd backend/

Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend
$ pwd
/c/Users/Paltos/Documents/Lab2_ISRPO_Leontev/backend
```

Шаг 4. Инициализация локального Git-репозитория

1. В папке **backend** выполните команду:

git init -b main

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend
$ git init -b main
Initialized empty Git repository in c:/Users/Paltos/Documents/Lab2_ISRPO_Leontev/backend/.git/

Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (main)
```

Пояснение

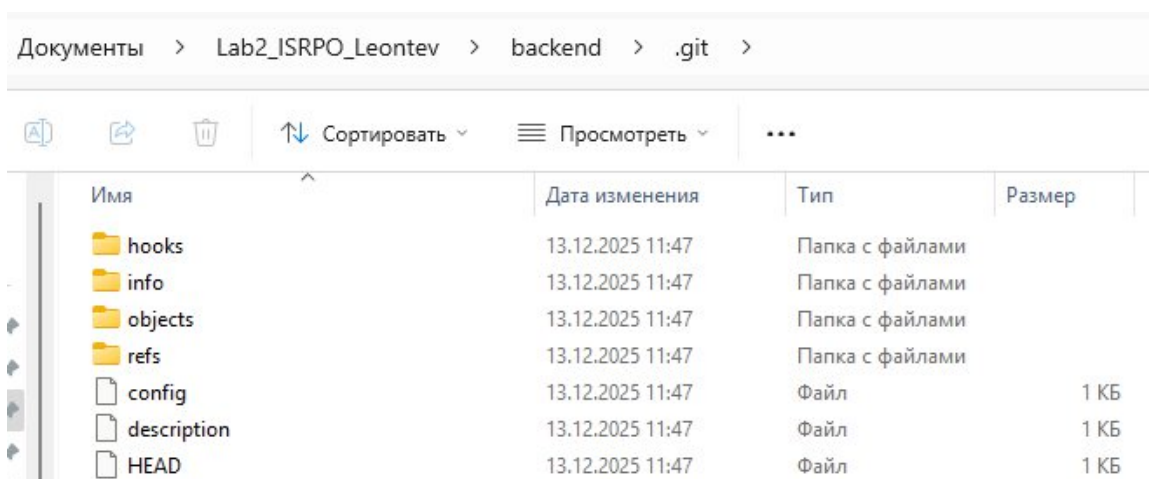
- Команда **git init** создаёт новый локальный Git-репозиторий.
- Параметр **-b main** указывает имя основной ветки.
- В современных версиях Git ветка **main** используется вместо устаревшего названия **master**.

После выполнения команды:

- папка **backend** становится Git-репозиторием;
- создаётся скрытая служебная папка **.git**, содержащая информацию о репозитории.

Шаг 5. Просмотр структуры Git-репозитория

1. Перейдите в проводнике в папку **backend**
2. Найдите и откройте скрытую папку **.git**.



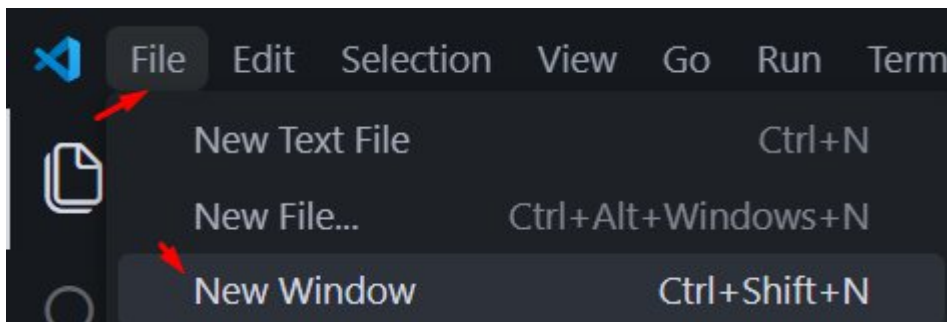
Внутри каталога .git хранятся файлы и директории, необходимые Git для:

- хранения истории изменений;
- управления ветками;
- отслеживания состояния файлов проекта.

Шаг 6. Подключение проекта в Visual Studio Code

1. Запустите **Visual Studio Code**.

2. Если открылся предыдущий проект, создайте новое окно: **File** → **New Window**:



3. Откройте папку с лабораторной работы №2 (Lab2_ISRPO_FIO).

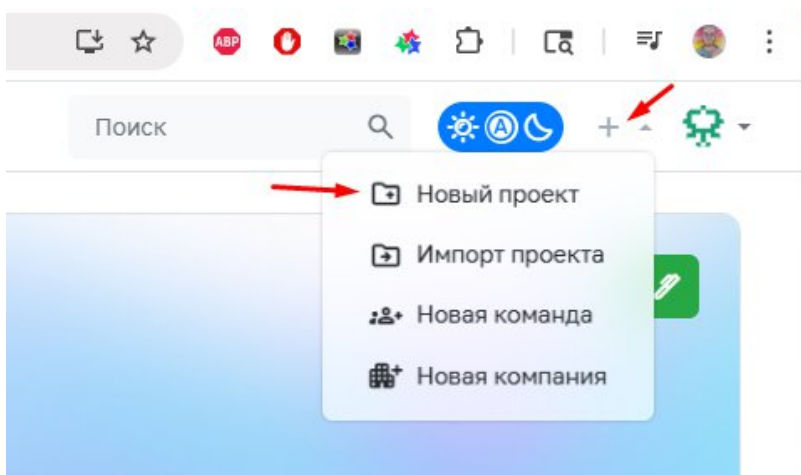
Шаг 7. Создание удалённого репозитория GitFlic и привязка Git

1. Перейдите на сайт сервиса **GitFlic**:

<https://gitflic.ru/>

2. Создайте новый репозиторий со следующими параметрами:

- **Название:** Lab2-Git_FIO
- **Язык программирования:** C#
- **Основная ветка:** main
- **Тип проекта:** публичный
- **Инициализация с README:** отключена



Создать проект

Репозиторий содержит все файлы проекта и историю их изменений

Новый проект Импорт проекта

Владелец

Denis Leontev

Название проекта

Lab2-Git_Leontev

Язык программирования

C#

Задать имя стандартной ветке

main

URL Проекта

https://gitflic.ru/project/paltosik92/lab2-git_leontev

Описание

Описание

Инициализировать этот проект с:

☐ Добавить README.md файл

Файл, в котором вы можете добавить описание своего проекта

Добавить .gitignore шаблон

Перечень файлов, которые будут игнорироваться Git

Вид шаблона

Добавить конфигурацию CI/CD конвейеров

Файл, в котором определены правила и этапы работы конвейеров

Вид шаблона

☒ Публичный проект

Любой пользователь интернета может увидеть этот репозиторий. Вы сами выбираете кто сможет делать коммиты в этот репозиторий.

☐ Приватный проект

Вы сами выбираете кто может увидеть этот репозиторий и кто сможет делать в нем коммиты.

Создать проект

3. После создания репозитория внизу страницы будут показаны команды для подключения существующего проекта:

Запустить существующий репозиторий

```
cd existing_folder
git remote rename origin old-origin
git remote add origin https://gitflic.ru/project/paltosik92/lab2-git_leontev.git
git push -u origin --all
git push -u origin --tags
```

4. Откройте терминал в **Visual Studio Code** и убедитесь, что вы находитесь в папке **backend**:

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev
$ cd ./backend/

Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (main)
```


5. Привяжите удалённый репозиторий к локальному:

git remote add origin ссылка_на_репозиторий

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab1_ISRPO_Leontev (master)
$ git remote add origin https://github.com/Paltosik92/Lab1-Fullstack_Leontev.git
```

6. Проверьте подключение удалённого репозитория:

git remote -v

В результате будут отображены адреса удалённого репозитория для операций получения (**fetch**) и отправки (**push**) изменений, например:

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (main)
$ git remote -v
origin https://gitflic.ru/project/paltosik92/lab2-git_leontev.git (fetch)
origin https://gitflic.ru/project/paltosik92/lab2-git_leontev.git (push)
```

Это означает, что локальный репозиторий успешно связан с удалённым.

Шаг 8. Наполнение backend

1. Создание консольного приложения .NET

В папке **backend** создадим консольное приложение с помощью .NET CLI:

dotnet new console -o Program --use-program-main

Пояснение команды

- *dotnet new console* — создаёт новый шаблон консольного приложения.
- *-o Program* — указывает имя папки, в которой будет создан проект.
- *--use-program-main* — создаёт классическую точку входа Main, а не упрощённый вариант с программой верхнего уровня.

2. Созданные файлы

После выполнения команды будут созданы следующие файлы:

- **Program.cs** — основной файл с исходным кодом консольного приложения и методом Main.
- **Program.csproj** — файл конфигурации проекта (XML-разметка), содержащий:
 - информацию о версии .NET;
 - тип проекта;
 - зависимости и параметры сборки.

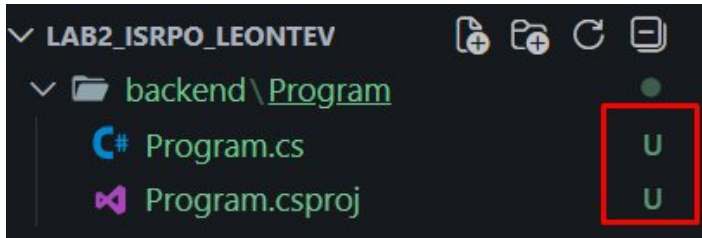
Файл **.csproj** — это XML-файл, описывающий конфигурацию проекта.

3. Статус файлов в Git

В панели контроля версий Visual Studio Code новые файлы будут помечены буквой **U (Untracked)** и подсвечены зелёным цветом.

Это означает, что Git:

- видит файлы;
- но они ещё не добавлены в индекс (staging area) и не отслеживаются репозиторием.

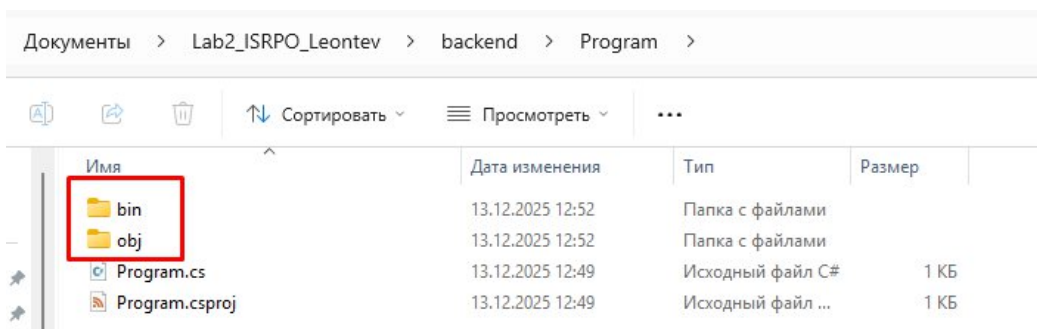


4. Папки bin и obj

После создания проекта .NET автоматически создаёт служебные папки:

- **bin** — содержит скомпилированные файлы;
- **obj** — содержит временные файлы сборки.

В Visual Studio Code эти папки не видны, так как их отображение было отключено в предыдущей лабораторной работе. Однако в проводнике операционной системы они присутствуют.



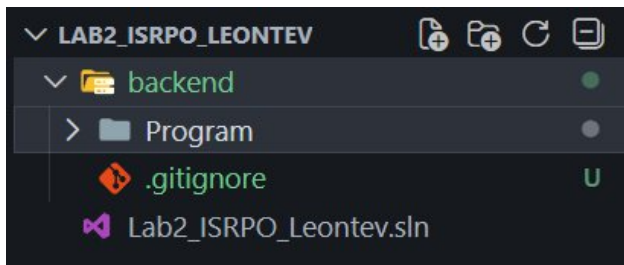
Данные папки **не должны храниться в Git-репозитории**, так как они могут быть восстановлены автоматически.

Шаг 9. Создание файла .gitignore

Для исключения служебных файлов и папок из репозитория создадим файл **.gitignore**

1. В папке **backend** создайте файл:

.gitignore



2. Добавьте в него следующее содержимое:

bin/

obj/

.vs/

Файл `.gitignore` указывает Git, какие файлы и каталоги необходимо игнорировать.

Шаг 10. Добавление файлов в Git и отправка на сервер

1. Проверьте состояние репозитория:

git status

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (main)
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        Program/
```

2. Добавьте все файлы (кроме игнорируемых) в индекс:

git add .

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (main)
$ git add .
```

Снова проверим состояние репозитория:

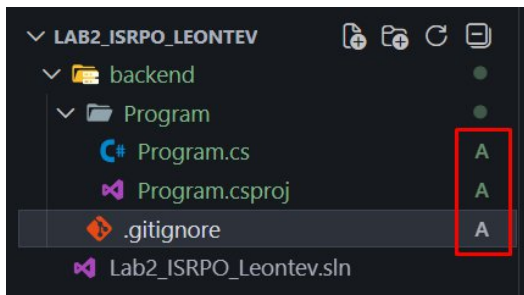
```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (main)
$ git status

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   .gitignore
        new file:   Program/Program.cs
        new file:   Program/Program.csproj
```


Добавлены три файла: **.gitignore** в корне, а также **Program.cs** и **Program.csproj** внутри папки **Program**.

Также в **Explorer** у нас у наших файлов появился значок A (**Added**):



3. Создайте коммит с описанием изменений:

git commit -m "Initial commit: create console app"

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (main)
$ git commit
[main (root-commit) b12c280] Initial commit: create console app
3 files changed, 22 insertions(+)
create mode 100644 .gitignore
create mode 100644 Program/Program.cs
create mode 100644 Program/Program.csproj
```

Результат коммита:

- **[main]** - коммит создан в ветке main
- **(root-commit)** - **важный момент!** Это самый первый коммит в репозитории (корневой коммит)
- **b12c280** - хеш коммита (уникальный идентификатор, первые 7 символов из 40)
- **Initial commit: create console app** - сообщение коммита

Изменения в файлах

1. Общая статистика:

- 3 files changed - изменилось 3 файла
- 22 insertions(+) - добавлено 22 строки кода
- Нет удалений (-), так как это первый коммит

2. Информация о каждом файле:

- .gitignore - файл для игнорирования ненужных файлов в Git
- Program/Program.cs - основной файл с кодом C#
- Program/Program.csproj - файл проекта C#

3. Режимы файлов (mode 100644):

- 100644 - стандартные права доступа в Git:
 - 100 - обычный файл
 - 644 - права доступа: владелец может читать/писать, остальные - только читать

4. create mode - файлы созданы впервые (не изменены существующие)

4. Теперь введём команду для показа истории коммитов в текущей ветке:

git log

```
commit b12c280b3caeec485c89849b52e9dfc94d71e3d3 (HEAD -> main)
```

Полный хеш коммита: `b12c280b3caeec485c89849b52e9dfc94d71e3d3` (40 символов)

- (HEAD -> main):
 - HEAD - указатель на текущий коммит
 - main - текущая ветка
 - Это означает, что вы находитесь на коммите b12c280 в ветке main

Автор коммита:

```
Author: Paltosik92 <denis.leontev92@gmail.com>
```

Дата и время создания коммита:

```
Date: Sat Dec 13 13:04:08 2025 +0300
```

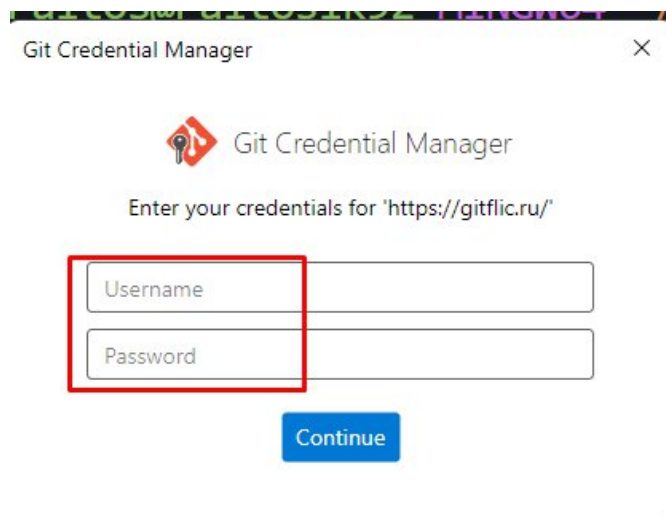
Сообщение коммита:

```
Initial commit: create console app
```

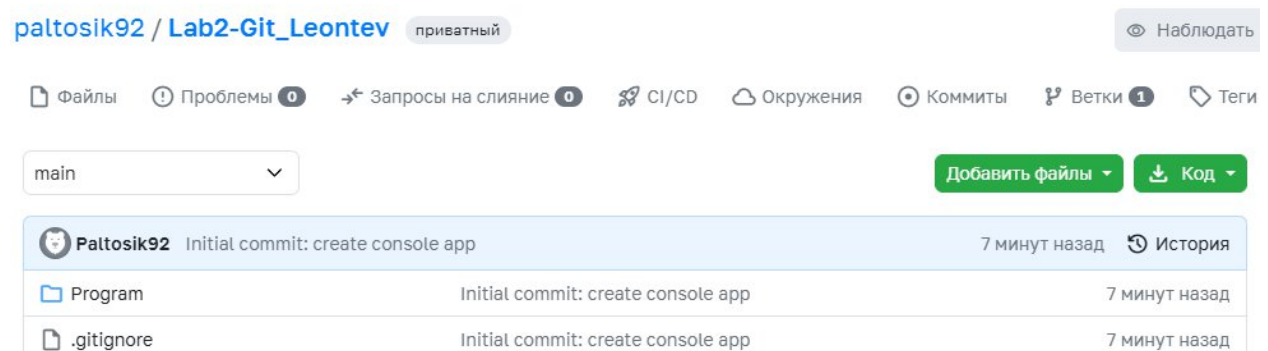
5. Отправьте изменения в удалённый репозиторий:

`git push -u origin main`

У вас должно появиться окно, где вам нужно будет ввести данные учётный записи от аккаунта:



После успешного выполнения команды файлы проекта будут загружены в удалённый репозиторий GitFlic:



Вывод в терминале:

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (main)
$ git push -u origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 16 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 643 bytes | 643.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Updating references: 100% (1/1)
To https://gitflic.ru/project/paltosik92/lab2-git_leontev.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Разбор вывода по строкам:

1. Подготовка объектов: Enumerating objects: 6, done.

- Git анализирует, какие объекты нужно отправить
- **6 объектов** - это коммит + файлы + деревья (tree objects)

2. Подсчет объектов: Counting objects: 100% (6/6), done.

- Подтверждение: все 6 объектов будут отправлены

3. Сжатие данных: Delta compression using up to 16 threads

Compressing objects: 100% (5/5), done.

- Используется многопоточное сжатие (до 16 потоков)
- 5 из 6 объектов сжаты (коммиты обычно не сжимаются дополнительно)
- **Delta compression** - В первом коммите дельт нет, но в будущем Git будет отправлять только различия между версиями

4. Запись и отправка: Writing objects: 100% (6/6), 643 bytes | 643.00 KiB/s, done.

- Все 6 объектов записаны и отправлены
- Общий размер: 643 байта
- Скорость: 643 КБ/с

5. Итоговая статистика: Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)

- **Total 6** - всего 6 объектов отправлено
- **delta 0** - нет дельт (изменений), так как это первый коммит
- **reused 0** - объекты не переиспользованы
- **pack-reused 0** - пакеты не переиспользованы

6. Обновление ссылок на сервере: remote: Updating references: 100% (1/1)

- На удаленном сервере обновляются ссылки (в данном случае одна ветка main)

7. Куда отправлено: To https://gitflic.ru/project/paltosik92/lab2-git_leontev.git

- **URL удаленного репозитория:** GitFlic
- **Путь к проекту:** /project/paltosik92/lab2-git_leontev.git

8. Создание ветки: * [new branch] main -> main

- [new branch] - создана новая ветка на удаленном репозитории
- main -> main - локальная ветка main отправлена в удаленную ветку main

9. Настройка отслеживания: branch 'main' set up to track 'origin/main'.

- **Самая важная часть флага -u!**
- Локальная ветка main теперь отслеживает удаленную ветку origin/main
- Теперь можно использовать просто git push и git pull без указания параметров

Шаг 10. Внесение изменений и создание второго коммита

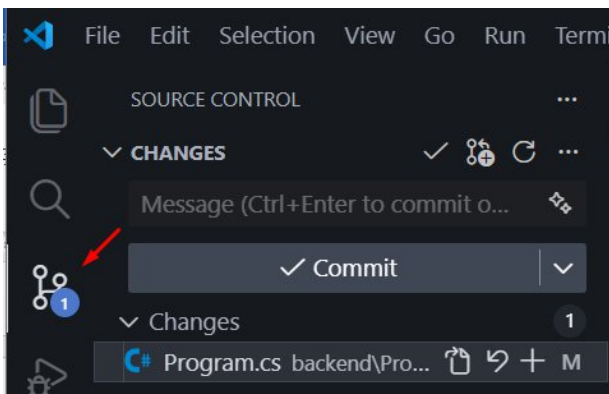
Теперь изменим код программы и увидим, как Git реагирует на правки.

1. Откройте файл **Program.cs**.
2. Добавьте ввод имени пользователя и измените вывод программы на персонализированное приветствие, например:

```
7 Console.WriteLine("Enter your name:");
8 string? name = Console.ReadLine();
9 Console.WriteLine($"Hello, {name}");
```

3. Обратите внимание на левую часть редактора кода:
 - о рядом с изменёнными строками появится **вертикальная линия**;
 - о она показывает, какие строки были изменены по сравнению с последним коммитом.

Также в разделе **Source Control** Visual Studio Code файл будет отмечен как изменённый.



Шаг 11. Создание второго коммита через терминал и просмотр истории

1. Откройте терминал в Visual Studio Code.
2. Проверьте состояние репозитория:

git status

Git покажет, что файл Program.cs был изменён:

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2
$ git status
On branch main
Your branch is up to date with 'origin/main'

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   Program/Program.cs
```


3. Добавьте изменения в индекс:

git add Program/Program.cs

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (main)
$ git add Program/Program.cs
```

4. Создайте новый коммит:

git commit -m "Add user name input and greeting"

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (main)
$ git commit -m "Add user name input and greeting"
[main bff4948] Add user name input and greeting
1 file changed, 4 insertions(+), 2 deletions(-)
```

5. Просмотрите историю коммитов:

git log

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (main)
$ git log
commit bff494859ed2884932d9c47d1caf04ea10b3618f (HEAD -> main)
Author: Paltosik92 <denis.leontev92@gmail.com>
Date: Sat Dec 13 13:35:28 2025 +0300

    Add user name input and greeting

commit b12c280b3caeec485c89849b52e9dfc94d71e3d3 (origin/main)
Author: Paltosik92 <denis.leontev92@gmail.com>
Date: Sat Dec 13 13:04:08 2025 +0300

    Initial commit: create console app
```

В выводе команды git log отображается:

- хеш коммита;
- автор;
- дата;
- сообщение коммита.

Таким образом можно отследить всю историю изменений проекта.

Шаг 12. Создание коммита через Visual Studio Code

Теперь выполним аналогичные операции, но используя интегрированные инструменты **Visual Studio Code**. Этот подход более наглядный и удобен для начинающих.

1. Вносим изменения в код

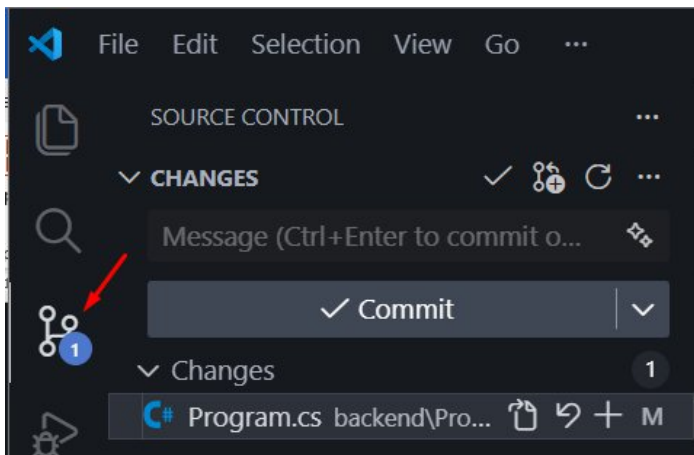
Откройте файл **Program.cs** и добавьте ещё одну строку с выводом дополнительного сообщения, например:


```
10 | Console.WriteLine($"Hello, {name}");  
Console.WriteLine("How are you?");
```

После сохранения файла изменения будут автоматически обнаружены системой контроля версий.

2. Открываем панель контроля версий

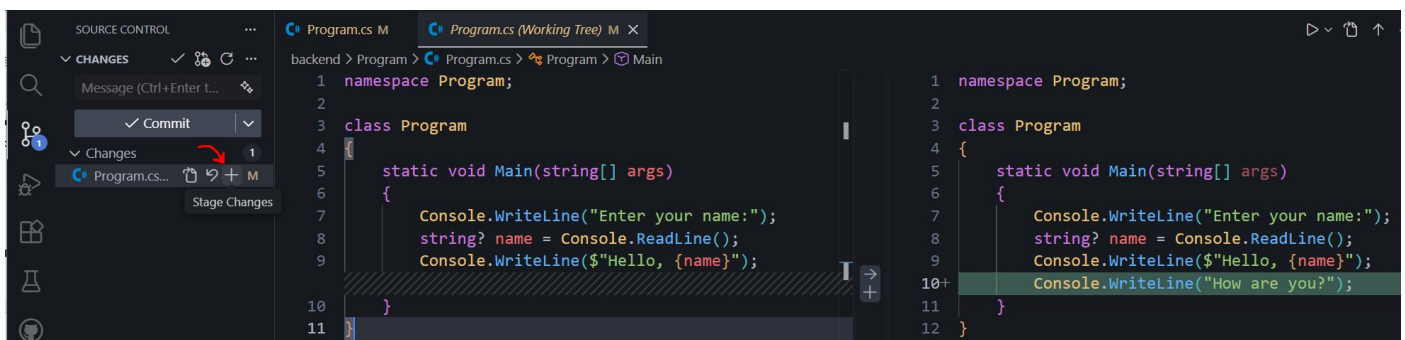
Перейдите во вкладку **Source Control** (значок ветки в левой панели или **Ctrl+Shift+G**). Здесь отображаются все измененные файлы:



3. Подготавливаем изменения (Stage)

В списке изменений:

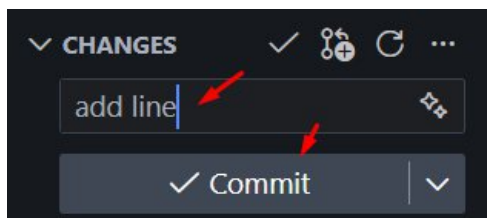
- Убедитесь, что файл Program.cs отображается в разделе **Changes** (изменения)
- Нажмите значок + (Stage Changes) рядом с файлом или используйте кнопку + над списком для всех файлов



"Staging" (подготовка) позволяет выбрать, какие именно изменения попадут в коммит. Это полезно, когда вы меняли несколько файлов, но хотите закоммитить их отдельно.

4. Создаем коммит

1. Введите осмысленное сообщение коммита в текстовое поле сверху, например:
"add line: дополнительное сообщение в консоль"
2. Нажмите кнопку **Commit** (галочка над полем сообщения)

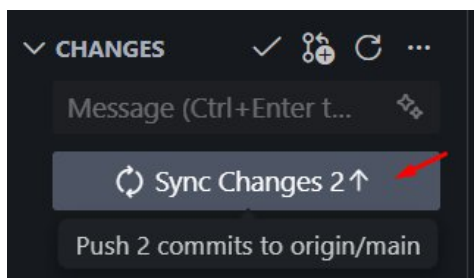


Теперь ваши изменения сохранены в локальном репозитории как новый коммит.

5. Синхронизация с удаленным репозиторием

Для отправки изменений на сервер **GitFlic**:

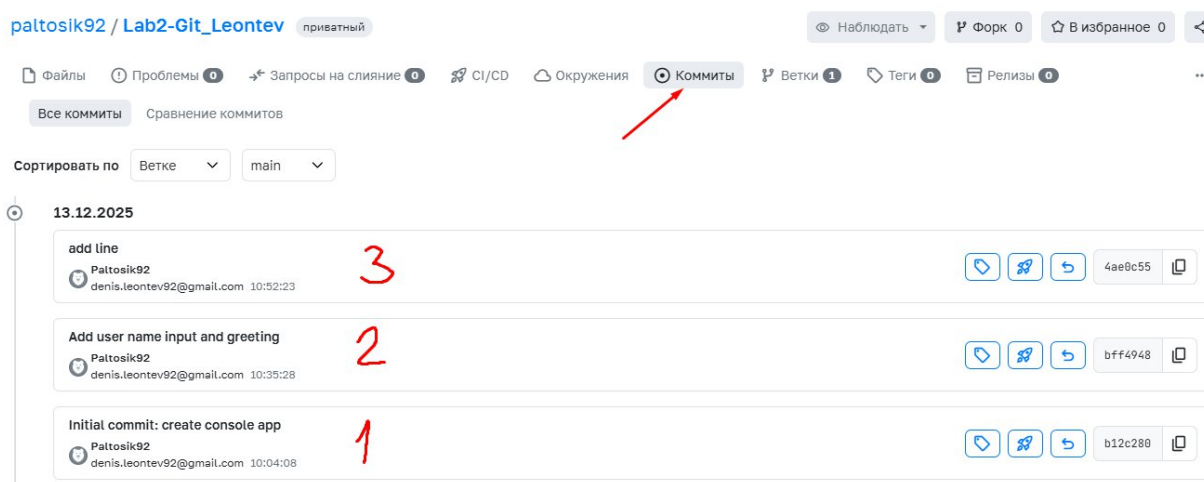
- Нажмите кнопку **Sync Changes** в нижней панели VS Code



VS Code выполнит команды `git push` и при необходимости `git pull`, чтобы синхронизировать вашу локальную копию с удаленным репозиторием.

6. Проверка на GitFlic

Перейдите в ваш репозиторий на **GitFlic** во вкладку **Коммиты**.



Теперь вы увидите три коммита. Каждый коммит содержит полную информацию: автора, дату, изменения в коде и хеш-идентификатор.

Шаг 13. Установка расширений Git для Visual Studio Code: Улучшаем рабочий процесс

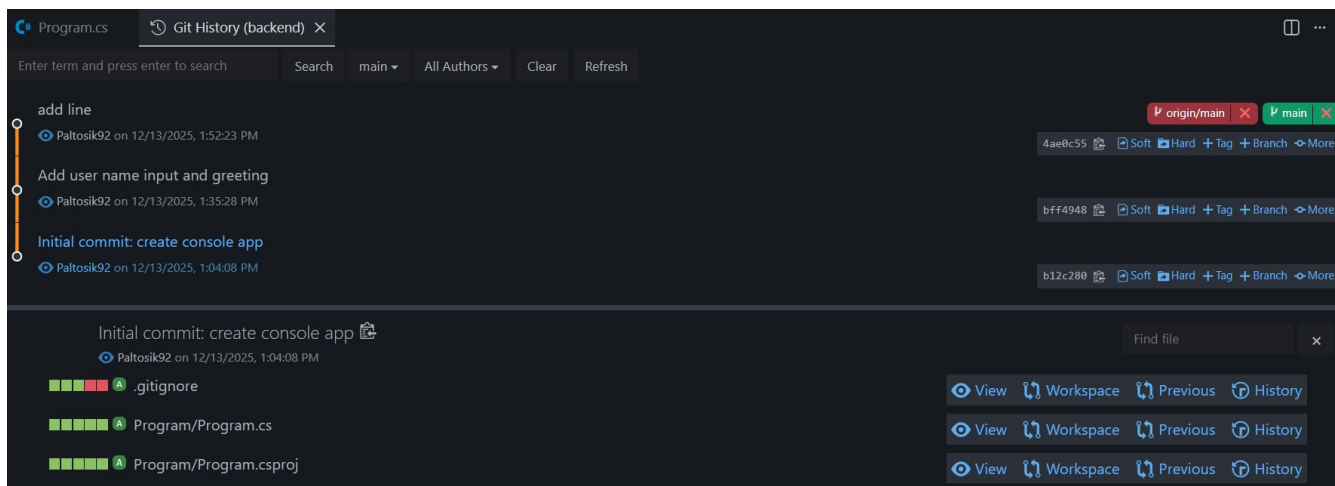
Для профессиональной работы с Git в VS Code рекомендуется установить несколько расширений, которые сделают процесс более наглядным и эффективным.

Git History: Детальный просмотр истории

Это расширение позволяет глубоко анализировать историю изменений прямо в редакторе.

Как использовать:

1. Откройте **Command Palette** (**Ctrl+Shift+P**)
2. **Введите:** Git: View History (git log)
3. Откроется специальная вкладка с полной историей коммитов



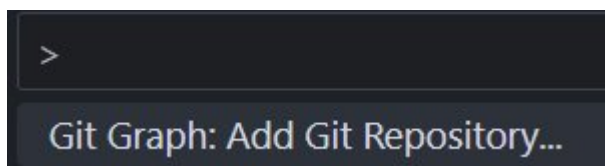
Преимущества: Можно просматривать diff (различия) между любыми версиями файлов, откатывать изменения, анализировать кто и когда вносил правки.

Git Graph: Визуализация ветвления

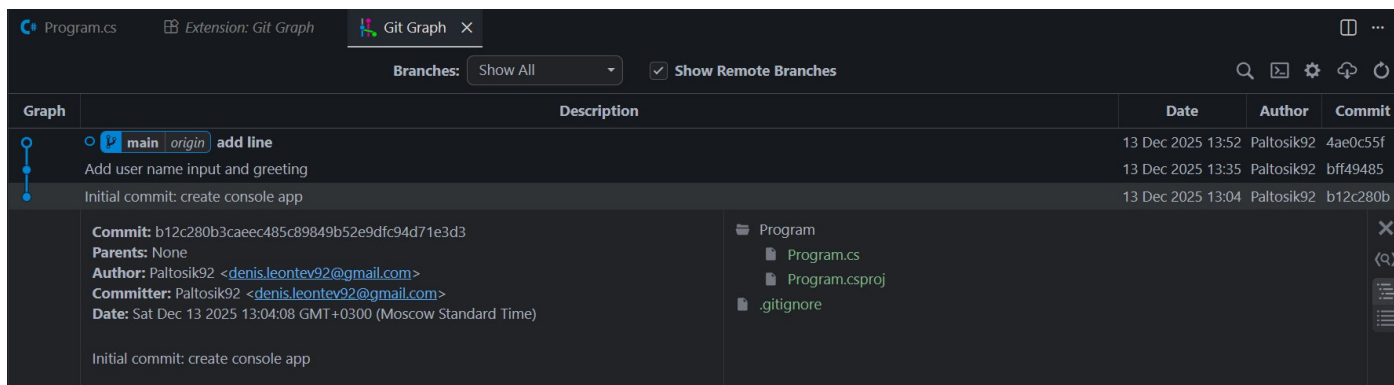
Отображает историю репозитория в виде понятного графа веток — незаменимый инструмент для работы с ветвлениями.

Настройка:

1. В **Command Palette** введите: Git Graph: Add Git Repository

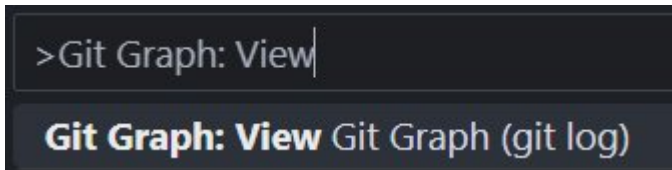


2. Выберите папку вашего проекта (ту, где находится скрытая директория **.git!**)
3. Откроется вкладка **Git Graph** с визуальным представлением

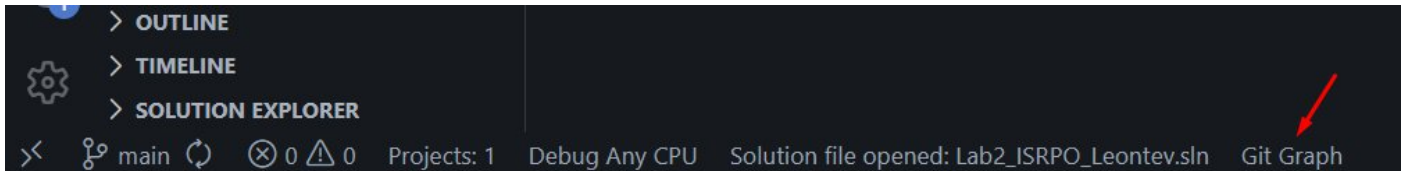


Альтернативный запуск:

- Используйте команду Git Graph: View Git Graph (git log) из Command Palette



- Или найдите вкладку Git Graph в левой панели VS Code



Особенность: Граф наглядно показывает отношения между коммитами, ветками и тегами, что особенно полезно при слияниях.

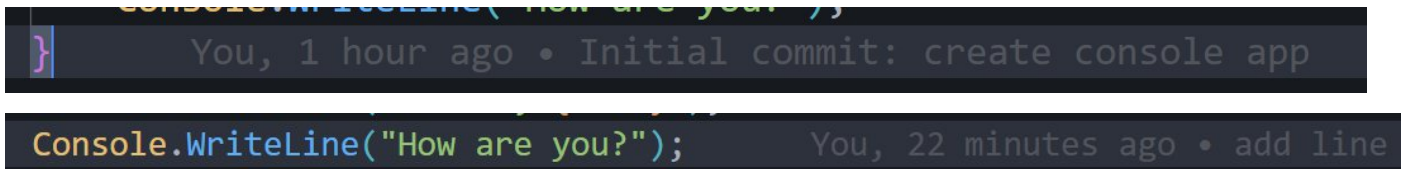
GitLens: Суперсила для разработчика

Мощнейшее расширение, которое превращает VS Code в профессиональную Git-станцию.

Ключевые возможности после установки:

Аннотации к коду:

- Наведение на строку показывает: кто, когда и в каком коммите её менял
- Пример: видите комментарий вида:



Детали коммитов:

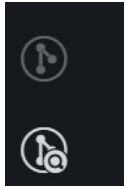
- При наведении на строку с Console.WriteLine увидите полное сообщение коммита
- Быстрый доступ к diff между версиями

Расширенный поиск:

- Поиск по истории коммитов, авторам, изменениям
- Анализ влияния изменений

Панели GitLens:

- В левой панели появятся две новые вкладки:
 - GitLens** — расширенная информация о репозитории
 - GitLens Inspect** — детальный анализ текущего состояния



Шаг 14. Углубляемся в историю: продвинутые команды Git log и show

Теперь, когда у нас есть несколько коммитов, давайте научимся эффективно анализировать историю изменений с помощью мощных команд Git.

1. Стандартный просмотр истории

Введите в терминале команду:

git log

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (main)
$ git log
commit 4ae0c55fb96acd6020b2e33e877ae3a102bf5000 (HEAD -> main, origin/main, origin/HEAD)
Author: Paltosik92 <denis.leontev92@gmail.com>
Date: Sat Dec 13 13:52:23 2025 +0300

    add line

commit bff494859ed2884932d9c47d1caf04ea10b3618f
Author: Paltosik92 <denis.leontev92@gmail.com>
Date: Sat Dec 13 13:35:28 2025 +0300

    Add user name input and greeting

commit b12c280b3caeec485c89849b52e9dfc94d71e3d3
Author: Paltosik92 <denis.leontev92@gmail.com>
Date: Sat Dec 13 13:04:08 2025 +0300

    Initial commit: create console app
```

Что показывает:

- Полные хеши коммитов
- Авторы и их email
- Даты создания
- Сообщения коммитов

Важная техническая деталь: Выход из просмотра

Если терминал показывает : в нижнем углу и не возвращает управление — это **pager** (программа постраничного вывода, часто less).

Как выйти:

- Нажмите английскую букву q (quit)
- Или Ctrl+C (если q не работает)



2. Компактный формат

Следующая команда:

git log --oneline

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (main)
$ git log --oneline
4ae0c55 (HEAD -> main, origin/main, origin/HEAD) add line
bff4948 Add user name input and greeting
b12c280 Initial commit: create console app
```

Для быстрого просмотра, когда нужны только основные данные.

Особенности:

- Показывает сокращенный хеш (первые 7 символов)
- Только сообщение коммита
- Одна строка на коммит — удобно для быстрой навигации

3. Детальный разбор конкретного коммита

Самая информативная команда, которая показывает ВСЕ детали конкретного коммита.

Что включает вывод:

1. Полная информация о коммите:

- о Полный хеш
- о Ссылки на ветки (HEAD, origin и т.д.)
- о Автора и дату
- о Сообщение

2. Diff (различия):

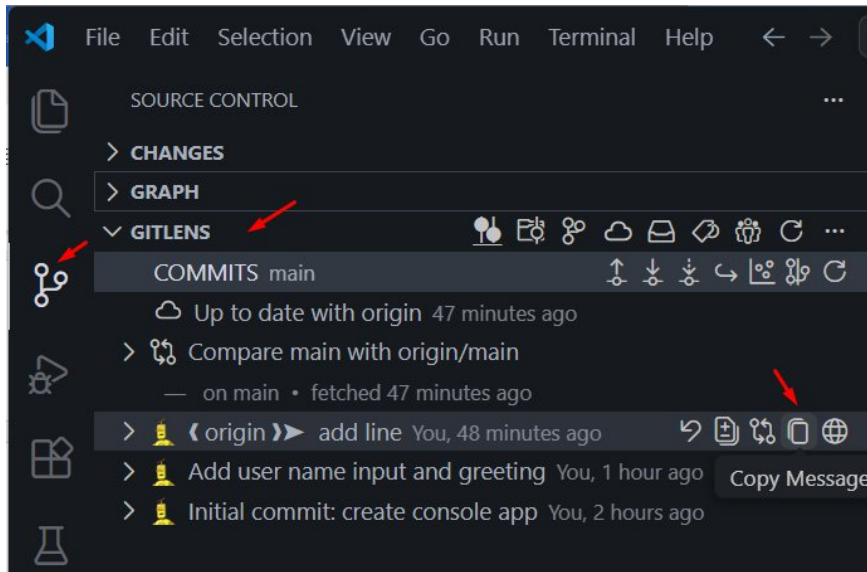
- о Какие файлы изменены
- о Конкретные строки, которые были добавлены/удалены
- о Контекст изменений

Простой способ копирования хеша

Через GitLens (рекомендуется):

1. Откройте панель **Source Control**
2. Перейдите во вкладку **GitLens**
3. Найдите нужный коммит в истории

4. Наведите курсор на коммит → появится кнопка **Copy SHA** (полный хеш)
5. Или **Copy Short SHA** (сокращенный хеш)



Далее вводим команду (вы вводите свой хеш):

git show ваш_хеш

```
$ git show 4ae0c55fb96acd6020b2e33e877ae3a102bf5000
commit 4ae0c55fb96acd6020b2e33e877ae3a102bf5000 (HEAD -> main, origin/main, origin/HEAD)
Author: Paltosik92 <denis.leontev92@gmail.com>
Date: Sat Dec 13 13:52:23 2025 +0300

    add line

diff --git a/Program/Program.cs b/Program/Program.cs
index 1529cdb..1089c65 100644
--- a/Program/Program.cs
+++ b/Program/Program.cs
@@ -7,5 +7,6 @@ class Program
     Console.WriteLine("Enter your name:");
     string? name = Console.ReadLine();
     Console.WriteLine($"Hello, {name}");
+    Console.WriteLine("How are you?");
 }
\ No newline at end of file
```

Самая важная часть: Diff (различия)

- --- a/Program/Program.cs — исходный файл (до изменений)
- +++ b/Program/Program.cs — измененный файл (после)
- @@ -7,5 +7,6 @@ — контекст изменений (строки 7-12)
- + (зеленый в терминале) — добавленная строка
 - - (красный в терминале) — удаленная строка (в нашем случае нет)

Шаг 15. Работа с ветками: создание, переключение и управление

Ветвление — одна из самых мощных возможностей Git. Давайте разберемся, зачем это нужно и как эффективно работать с ветками.

Зачем нужны ветки?

Основные сценарии использования:

1. **Разработка нового функционала** — изолированные изменения без риска сломать основную версию
2. **Эксперименты** — тестирование идей без влияния на стабильный код
3. **Исправление багов** — срочные фиксы в отдельном потоке
4. **Параллельная работа команды** — каждый разработчик работает в своей ветке
5. **Версионирование** — поддержка разных версий продукта

Простая аналогия: Представьте, что вы пишете книгу. `main` — это готовые главы, а ветки — черновики новых глав, которые можно редактировать независимо.

Команды для работы с ветками

1. Создание новой ветки с переключением

git checkout -b new-feature

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (main)
$ git checkout -b new-feature
Switched to a new branch 'new-feature'

Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (new-feature)
```

Что делает:

- Создает ветку **new-feature**
- Немедленно переключается на неё
- **Устаревший подход** (но еще широко используется)

2. Создание ветки без переключения

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (new-feature)
$ git branch hotfix
```

Что делает:

- Создает ветку **hotfix**
- **Остается в текущей ветке** — нужно переключаться отдельно

3. Просмотр всех веток

git branch

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (new-feature)
$ git branch
  hotfix
  main
* new-feature
```

* — текущая активная ветка

Современный подход: git switch

С Git 2.23+ появились более понятные команды:

Старая команда	Новая команда	Назначение
git checkout -b	git switch -c	Создать и переключиться
git checkout	git switch	Просто переключиться

Выполним команды.

Создание и переключение

git switch -c feature

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (new-feature)
$ git switch -c feature
Switched to a new branch 'feature'

Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (feature)
```

Переключение на существующую ветку

git switch main

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (feature)
$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (main)
```

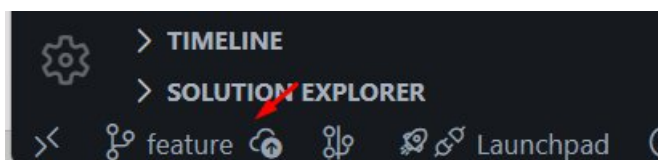
Переключение на предыдущую ветку

git switch -

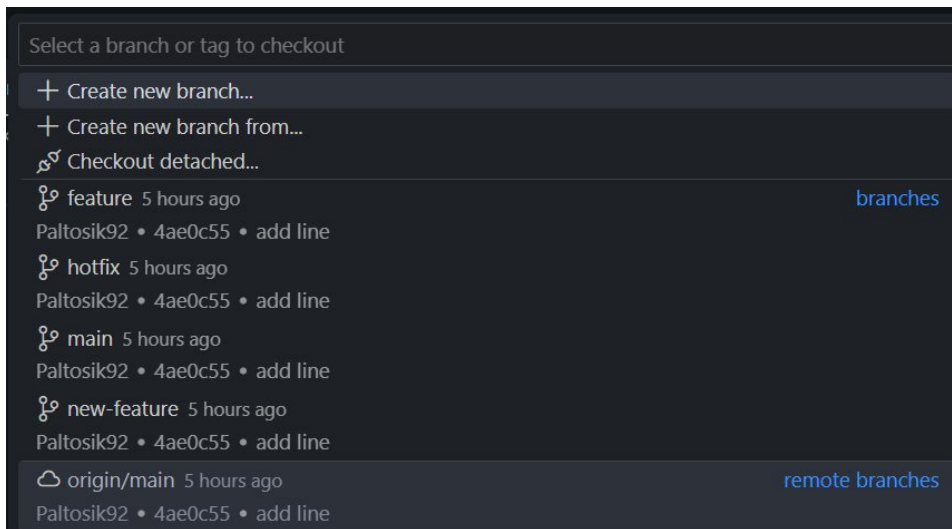
```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (main)
$ git switch -
Switched to branch 'feature'

Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (feature)
```

Также в Visual Studio Code мы можем нажать на значок внизу с ветками:



И выбрать на какую ветку мы хотим переключиться:



Преимущества git switch:

- Более интуитивный синтаксис
- Меньше путаницы с другими функциями **checkout**
- Лучшие сообщения об ошибках

4. Удаление веток

Безопасное удаление (если изменения слиты)

Давайте удалим ветку **hotfix**. Проверим что мы не на этой ветке:

git branch

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (feature)
$ git branch
* feature
  hotfix
  main
  new-feature
```

Удаляем ветку командой:

git branch -d hotfix

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (feature)
$ git branch -d hotfix
Deleted branch hotfix (was 4ae0c55).

Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (feature)
$ git branch
* feature
  main
  new-feature
```

Шаг 16. Работа с удаленным репозиторием (GitFlic): от ветки до слияния

1. Вносим изменения в новой ветке

Сначала убедимся, что мы находимся в ветке **feature**:

git switch feature

Теперь добавим новую функциональность в Program.cs. Например:

```
10 Console.WriteLine("How are you?");  
11 + Console.WriteLine(1+1);
```

2. Коммитим изменения

Сохраняем изменения в локальный репозиторий:

git add .

git commit -m "new branch"

3. Отправляем ветку на GitFlic

При попытке стандартного пуша (**git push**) Git подскажет правильную команду:

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (feature)  
$ git push  
fatal: The current branch feature has no upstream branch.  
To push the current branch and set the remote as upstream, use  
  
    git push --set-upstream origin feature  
  
To have this happen automatically for branches without a tracking  
upstream, see 'push.autoSetupRemote' in 'git help config'.
```

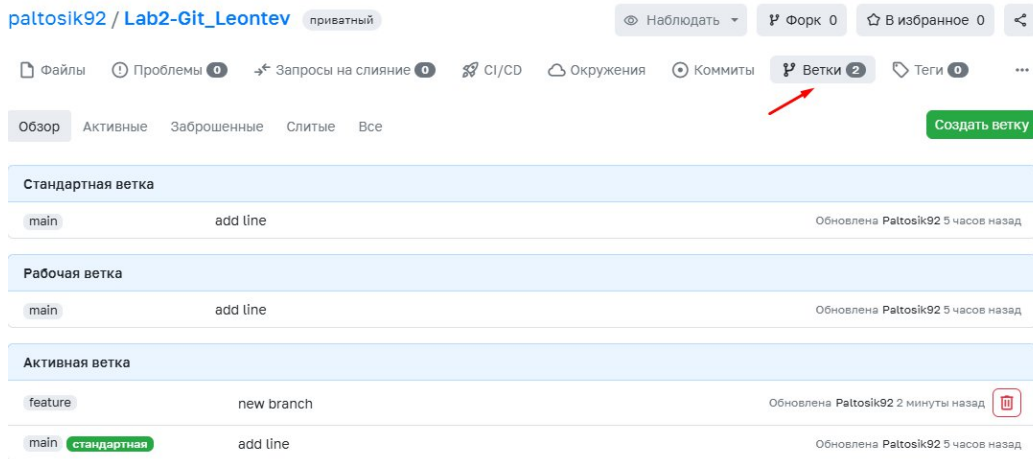
Выполняем предложенную команду и видим успешный вывод:

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (feature)  
$ git push --set-upstream origin feature  
Enumerating objects: 7, done.  
Counting objects: 100% (7/7), done.  
Delta compression using up to 16 threads  
Compressing objects: 100% (4/4), done.  
Writing objects: 100% (4/4), 403 bytes | 403.00 KiB/s, done.  
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)  
remote: Resolving deltas: 100% (1/1)  
remote: Updating references: 100% (1/1)  
To https://gitflic.ru/project/paltosik92/lab2-git_leontev.git  
* [new branch]      feature -> feature  
branch 'feature' set up to track 'origin/feature'.
```

4. Проверяем на GitFlic

Переходим на GitFlic во вкладку **Ветки** и видим две ветки:

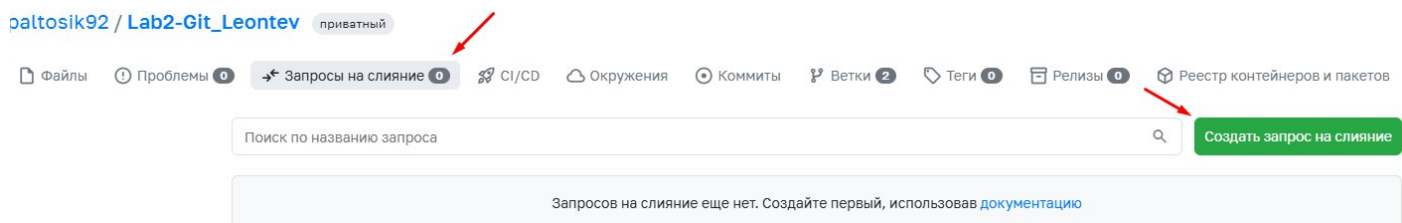
- **main** — основная ветка
- **feature** — наша новая ветка с изменениями



5. Создаем Pull Request (Запрос на слияние)

На **GitFlic** выполняем следующие шаги:

1. Переходим во вкладку **Запросы на слияние** (Pull Requests)
2. Нажимаем **Создать запрос на слияние**



Настраиваем параметры:

Создание запроса на слияние

Источник

paltosik92/lab2-gi

feature

Цель

paltosik92/lab2-gi

main

Заголовок

added line

Описание изменений

added line

Утверждающие

Выберите пользователя

Можно назначить до 15 человек

Рецензенты

Выберите пользователя

Можно выбрать до 10 человек

Лейблы

Выберите лейбл

Можно добавить до 20 лейблов

Связанные проблемы

Выберите проблему

Можно связать до 10 проблем

Заблокировать запрос

☐

Необходимо заполнить описание

☒ Удалить исходную ветку, после того как запрос на слияние будет принят

☒ Выполнить слияние одним коммитом (squash commit)

Отмена

Создать

- o **Источник:** ветка feature
- o **Цель:** ветка main
- o **Заголовок:** "added line"
- o **Описание:** added line
- o **Дополнительно:**
 - Удалить исходную ветку после слияния
 - Выполнить слияние одним коммитом (Squash)

4. Нажимаем **Создать**

6. Выполняем слияние

После проверки кода (если требуется ревью) и отсутствия конфликтов:

1. Нажимаем кнопку **Слить** (Merge)
2. Подтверждаем действие

#1 added line

[открыт](#) Denis Leontev хочет слить ветку paltosik92/lab2-git_leontev - feature в ветку main 1

[Обсуждения](#) [Коммиты](#) [Конвейеры](#) [Изменения](#)

Описание изменений

added line

☒ Можно завершить слияние веток

- ☒ Запрос на слияние не заблокирован
- ☒ Права на слияние
- ☒ Запрос на слияние без конфликтов
- ☒ Нет файлов, заблокированных другими пользователями

Слить ▾

Также вы можете воспользоваться командной строкой или вашей IDE для закрытия данного запроса.

Результат на GitFlic:

- Изменения из feature перенесены в main
- Ветка feature автоматически удалена
- Остается только одна ветка main

[Набл](#)

[Коммиты](#)

[Ветки 1](#)

[1](#)

7. Обновляем локальный репозиторий.

Для начала переключимся на ветку **main**:

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (feature)
$ git switch main
Switched to branch 'main'
Your branch is behind 'origin/main' by 2 commits, and can be fast-forwarded.
(use "git pull" to update your local branch)
```

Обновляем **main** командой:

git pull

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (main)
$ git pull
Updating 4ae0c55..07b825d
Fast-forward
 Program/Program.cs | 1 +
 1 file changed, 1 insertion(+)
```

Далее удалим неиспользуемые ветки и проверим список оставшихся:

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (main)
$ git branch -D feature
Deleted branch feature (was aacc07e).

Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (main)
$ git branch -D new-feature
Deleted branch new-feature (was 4ae0c55).

Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (main)
$ git branch
* main
```

И получим актуальное состояние сервера:

git fetch --prune

```
Paltos@Paltosik92 MINGW64 ~/Documents/Lab2_ISRPO_Leontev/backend (main)
$ git fetch --prune
From https://gitflic.ru/project/paltosik92/lab2-git_leontev
 - [deleted]          (none)      -> origin/feature
```

- Загружает информацию об удаленных ветках
- **--prune** — удаляет ссылки на несуществующие удаленные ветки

Самостоятельные задания

Задание 1

Цель: Создать новый мини-проект с нуля и пройти все этапы работы с Git, которые изучались в лабораторной работе.

Часть 1: Подготовительный этап

1. Создайте новую папку для проекта

```
mkdir git-practice-[ВашаФамилия]
```

```
cd git-practice-[ВашаФамилия]
```

Сделайте скриншот терминала после выполнения команд.

Сохраните как: 1-terminal-init-FIO.png (указываете вашу фамилию)

Часть 2: Инициализация Git-репозитория

2. Инициализируйте Git-репозиторий

3. Проверьте статус

Сделайте скриншот с выводом `git status`.

Сохраните как: 2-git-status-FIO.png (указываете вашу фамилию)

Часть 3: Создание первого коммита

4. Создайте файл README.md

5. Добавьте файл и создайте коммит

6. Посмотрите историю коммитов

Сделайте скриншот с выводом `git log --oneline`.

Сохраните как: 3-first-commit-FIO.png (указываете вашу фамилию)

Часть 4: Работа с ветками

7. Создайте и переключитесь на новую ветку

8. Добавьте новый файл с любым кодом

9. Закоммитьте изменения

10. Проверьте ветки

Сделайте скриншот с выводом `git branch`.

Сохраните как: 4-branches-FIO.png (указываете вашу фамилию)

Часть 5: Подключение удаленного репозитория

11. Создайте репозиторий на GitFlic/GitHub

- Создайте новый публичный репозиторий git-practice-[ВашаФамилия]
- **Не добавляйте** README, .gitignore и лицензию

12. Подключите удаленный репозиторий

13. Отправьте код на GitFlic/GitHub

Сделайте скриншот успешного выполнения git push.

Сохраните как: 5-push-success-FIO.png (указываете вашу фамилию)

Часть 6: Создание и отправка второй ветки

14. Вернитесь в ветку feature

15. Отправьте ветку на сервер

Сделайте скриншот с выводом git branch -a.

Сохраните как: 6-all-branches-FIO.png (указываете вашу фамилию)

Задание 2

В рамках этого задания вы создадите полезную шпаргалку по основным командам Git, которую сможете использовать в будущем для быстрого доступа к ключевым командам.

1. Создание файла-шпаргалки

В корневой папке вашего проекта создайте файл **git-cheatsheet.txt**

2. Наполнение файла содержанием

В файле **git-cheatsheet.txt** опишите основные команды, которые разбирались в рамках лабораторной работы с кратким разъяснением.

3. Добавьте свои примеры из лабораторной

Включите конкретные команды, которые вы использовали в работе:

- Команды для работы с GitFlic/GitHub
- Примеры из ваших экспериментов
- Проблемы, с которыми столкнулись, и их решения